

Algoritmos y Estructuras de Datos III

Trabajo Práctico N° 3	Unidad 3
Modalidad: Semi -Presencial	Estratégica Didáctica: Trabajo individual
Metodología de Desarrollo: Det. docente	Metodología de Corrección: Via Classroom.
Carácter de Trabajo: Obligatorio – Con Nota	Fecha Entrega: A confirmar por el Docente.

LISTAS – LISTAS ENLAZADAS DOBLES y LISTAS STL.

MARCO TEÓRICO (COMPLEJIDAD ALGORÍTMICA):

Responder el siguiente cuestionario en función de la bibliografía Obligatoria.

1. Describir el Concepto de Lista

Una lista es una estructura de datos que organiza y almacena información de forma dinámica. A diferencia de un array, cuyo tamaño se debe conocer antes de compilar y cuyos elementos están separados en memoria por distancias fijas, una lista asigna la memoria necesaria de manera dinámica. Está formada por nodos; cada nodo contiene un dato (info) y uno o más punteros que conectan al nodo siguiente o anterior. Esto permite que los nodos se coloquen en cualquier lugar de la memoria, y que agregar o eliminar elementos afecte solo a esos nodos, sin mover el resto de los datos como sucede en un array.

2. Describir el Concepto de:

A. Lista Vinculada

Esta es una lista de nodos en los que cada nodo tiene un dato y un puntero que apunta al siguiente nodo. Para acceder a toda la lista, se utiliza un solo puntero llamado head. El último nodo se identifica porque su puntero next no apunta a ningún otro nodo, es decir, es nulo.

Se utilizan dos punteros fijos, head y tail, para acceder rápidamente al primer y al último nodo de la lista. Las operaciones para agregar un nuevo nodo al principio o al final de la lista, llamadas addToHead y addToTail, se pueden realizar muy rápidamente, en un tiempo constante.

Sin embargo, si se necesita eliminar el último nodo de la lista, es necesario recorrer toda la lista para encontrar el nodo que está justo antes del último, lo que lleva mucho más tiempo, especialmente si la lista es muy larga. Esto se debe a que no hay un puntero directo al nodo que precede al último, por lo que hay que buscarlo nodo por nodo.

B. Lista Doblemente Vinculada

Para resolver el problema de no tener acceso al nodo anterior en una lista simple, cada elemento de una lista doblemente vinculada tiene dos indicadores: uno que apunta al siguiente elemento y otro que apunta al elemento anterior. Esto permite acceder al último elemento de forma directa, sin tener que recorrer toda la lista. Por lo tanto, tanto la función para agregar un elemento al final como la función para eliminar un elemento del final se ejecutan en un tiempo muy rápido. La desventaja es que cada elemento necesita un poco más de espacio para almacenar el segundo indicador.

C. Lista Circular

En una lista circular, el último elemento no apunta a nada, sino que vuelve al primer elemento, formando un círculo. Cada elemento tiene un elemento que lo sigue, sin parar. Esto se utiliza cuando se necesita recorrer los elementos en círculo, como cuando varios procesos comparten algo y se turnan para usarlo.

La forma en que se hace esto es con un solo puntero que apunta al final de la lista. Desde ahí, se puede acceder al primer elemento de la lista, lo que permite agregar elementos al principio o al final de la lista muy rápido.

D. Lista Circular doblemente Vinculada

La lista doblemente circular es una estructura de datos que combina las características de la lista doblemente vinculada y la lista circular. Esta estructura forma dos anillos: uno hacia adelante a través de `next` y otro hacia atrás a través de `prev`. Esto permite insertar y eliminar nodos en cualquier extremo en tiempo constante, que es $O(1)$, y también recorrer la lista doblemente circular en ambas direcciones de forma eficiente. La lista doblemente circular es la variante más completa y sirve de base para la implementación de `std::list` en la STL de C++.

3. Que beneficios tiene usar <LIST> de la STL?

La STL utiliza una lista doblemente vinculada para implementar `std::list`. Esta lista tiene punteros a la cabeza y a la cola. Los beneficios principales de esto son:

- Puedes insertar y eliminar elementos en cualquier posición de la secuencia en tiempo constante, siempre que tengas un iterador en esa posición. Esto la hace mejor que otros contenedores, como `vector` o `deque`, cuando se trata de mover muchos datos.
- Puedes iterar en ambas direcciones, lo que significa que puedes moverte hacia adelante y hacia atrás en la lista. Esto es diferente a `forward_list`, que solo permite iterar hacia adelante.
- La lista es genérica, lo que significa que puedes almacenar cualquier tipo de dato en ella, incluyendo tipos que tú mismo definas. Solo necesitas asegurarte de que tengan un constructor por defecto.
- La lista tiene funciones especiales que no están disponibles en `vector`. Por ejemplo, puedes transferir elementos entre listas con `splice`, fusionar listas ordenadas con `merge`, eliminar duplicados con `unique`, y eliminar elementos con una condición con `remove_if`.

Sin embargo, hay algunas desventajas. No puedes acceder a un elemento en una posición específica de inmediato. En su lugar, debes iterar desde el inicio de la lista, lo que puede tomar tiempo. Además, la lista utiliza memoria extra para los punteros de enlace de cada nodo.

4. Enumerar y Describir las Operaciones básicas que Implementan las Listas.

Las operaciones básicas que se pueden realizar son las siguientes:

- Agregar al inicio: esto se hace con la función `addToHead`. Esta función crea un nuevo nodo, lo enlaza al primer nodo actual y luego actualiza la referencia de inicio. Esto se hace en un tiempo constante.
- Agregar al final: esto se hace con la función `addToTail`. Esta función crea un nuevo nodo, enlaza el último nodo a este nuevo nodo y luego actualiza la referencia de final. Esto también se hace en un tiempo constante.

— Eliminar desde el inicio: esto se hace con la función `deleteFromHead`. Esta función guarda el dato del primer nodo, mueve la referencia de inicio al siguiente nodo y luego libera el nodo anterior. Esto se hace en un tiempo constante.

— Eliminar desde el final: esto se hace con la función `deleteFromTail`. Si se trata de una lista simple, esta función requiere recorrer toda la lista hasta llegar al nodo anterior al final, por lo que el tiempo que tarda es proporcional a la cantidad de nodos en la lista. Sin embargo, si se trata de una lista doble, esta función puede acceder directamente al nodo anterior al final, por lo que se hace en un tiempo constante.

— Eliminar un nodo: esto se hace con la función `deleteNode`. Esta función recorre la lista en busca del nodo que contiene el valor que se desea eliminar. Si el nodo que se desea eliminar es el primer nodo, esta función se hace en un tiempo constante. Sin embargo, en el peor de los casos, esta función tiene que recorrer toda la lista, por lo que el tiempo que tarda es proporcional a la cantidad de nodos en la lista.

— Buscar un elemento: esto se hace con la función `isInList`. Esta función recorre la lista comparando cada nodo con el valor que se está buscando. Si encuentra el valor, devuelve `true`. Si recorre toda la lista sin encontrar el valor, devuelve `false`. Al igual que la función `deleteNode`, el tiempo que tarda esta función es constante en el mejor de los casos, pero proporcional a la cantidad de nodos en la lista en el peor de los casos.

5. Dar 2 formas de Manejo de Error a listas vacías

Si intentas eliminar un nodo de una lista que no tiene elementos, el programa puede quedar bloqueado. Hay dos formas de manejar esto.

La primera forma es utilizar la función `assert`. Esto se hace poniendo `assert` antes de intentar eliminar el nodo, para asegurarse de que la lista no esté vacía. Si la lista está vacía, el programa se detiene y muestra un mensaje de error. Esta forma es directa, pero también es un poco brusca porque no permite que el programa siga ejecutándose.

```
int IntSLList::deleteFromHead() {  
  
    assert(!isEmpty()); // termina el programa si la lista está vacía  
  
    int e1 = head->info;  
  
    // ...  
  
    return e1;  
  
}
```

La segunda forma es utilizar excepciones. Cuando intentas eliminar un nodo de una lista vacía, se lanza una excepción. Luego, en la parte del código que llamó a esta función, se puede capturar esta excepción y manejarla como se desee. Por ejemplo, puedes mostrar un mensaje, asignar un valor alternativo o detener el programa. Esta forma es más flexible porque permite al programa decidir qué hacer en ese momento.

```
int IntSLList::deleteFromHead() {  
    if (isEmpty())  
        throw("Empty");  
  
    int el = head->info;  
  
    // ...  
  
    return el;  
}  
  
// En el código llamador:  
  
try {  
    n = list.deleteFromHead();  
} catch(char *s) {  
    cerr << "Error: " << s << endl;  
}
```

MARCO PRÁCTICO

1. Una Clínica Local nos pide Implementar un Sistema para manejar Pacientes. Debe tener un menú que permita implementar una Lista que facilite: registrar, modificar, eliminar y listar pacientes.
2. Idem 1 pero Implementando Listas en STL